



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA



Master Universitario en Inteligencia Artificial, Reconocimiento de Formas e Imagen Digital
Universidad Politécnica de Valencia

Comparativa implementación Asistente de Voz: Local VS Cloud

TRABAJO HERRAMIENTAS Y APLICACIONES DE LA INTELIGENCIA ARTIFICIAL

Máster Universitario en Inteligencia Artificial, Reconocimiento de Formas e Imagen Digital

Andrés Pachecho,
Miquel Gómez

CAPÍTULO 1

Introducción

En el contexto de la vida cotidiana, los asistentes de voz se han convertido en herramientas muy usadas por su gran versatilidad. Desde los primeros *Hey Siri*, a los *Ok Google* o *Alexa*, los asistentes virtuales han llegado a multitud de hogares y ayudan en el día a día de diversas tareas.

Al principio, estos asistentes eran muy limitados y no tenían apenas capacidad de respuesta. Eran todo instrucciones predefinidas que seguían casi como `if else`. Las voces y las frases que estas podían decir estaban pre-grabadas, por lo que no se podían salir de ese pequeño conjunto de respuestas.

Hoy en día, esto ha cambiado bastante. De primeras, la llegada de los modelos de lenguaje, capaces de generalizar las respuestas que puede dar un sistema, soluciona las limitaciones de '*a qué puede contestar el sistema*', ahora es prácticamente a todo. Luego, si se le suma que los modelos de voz modernos son capaces de pronunciar cualquier frase, palabra o fonema en casi todos los idiomas, tenemos la receta perfecta para crear un sistema que responda a cualquier petición en segundos.

En este trabajo, se pretende abordar la creación de un sistema de preguntas y respuestas mediante un asistente de voz virtual. En su conjunto, el proyecto consiste en una investigación para analizar la viabilidad de desarrollar un asistente de voz propio, evaluando y confrontando el rendimiento, la facilidad de desarrollo y la complejidad de utilizar alternativas totalmente locales frente a soluciones basadas en modelos de diferentes proveedores *cloud* (en la nube) para cada una de las fases (transcripción, RAG con información local y web, y síntesis de voz).

A lo largo de las secciones se habla de la estructura que se ha seguido, con qué intenciones, con qué tecnologías se ha implementado y cómo se ha ajustado.

Uso de herramientas y LLMs

Para el desarrollo de este trabajo, se ha hecho uso de herramientas LLM en diferentes puntos. Primero, para la interfaz gráfica, se ha partido de una plantilla y se ha usado Copilot para que realice la integración con el resto de la app (archivo `app/app.py`). Luego, también se ha usado Claude Code, muy útil a la hora de resolver de conflictos entre versiones. Por último, para revisar la redacción de esta memoria, se ha hecho uso de Gemini.

Todo se ha hecho con la supervisión de los autores y con control total sobre lo que hacían estas herramientas.

Hardware utilizado

Los experimentos y la demo se han lanzado en una máquina personal: GPU NVIDIA RTX 5070 (12 GB VRAM), procesador AMD Ryzen 9 9000X, 64 GB de RAM y Ubuntu 24.04 LTS.

CAPÍTULO 2

Planteamiento del proyecto

Para poder implementar un asistente capaz responder preguntas de forma confiable como el que se plantea, hay que tener en cuenta diversos puntos clave.

Primero que todo, el sistema debe ser cómodo de usar e interactuar con él. Para esto, la opción más intuitiva es que funcione mediante voz.

Si se piensa en que situaciones se suelen usar estos asistentes, lo primero que viene a la mente son situaciones en las que no es posible hacer uso de tecnología, como al conducir, cocinar o incluso en la ducha. Son situaciones en las que se tienen las manos ocupadas y por lo tanto, no se puede hacer uso ni de dispositivos móviles, ni de mucho menos teclado y ratón. Por lo que hacer las preguntas mediante voz, es la solución perfecta.

Luego, este sistema tiene que poder interpretar la instrucción, lo que requiere de un sistema capaz de entender preguntas genéricas de cualquier ámbito. La tecnología que mejor responde a esta necesidad son los LLMs, no requieren de mayor presentación. Ahora, estos traen consigo algunos problemas por su misma naturaleza, haciendo que su uso no sea simple: estos modelos no tienen información de nuestro contexto concreto, no tienen información actualizada del mundo en tiempo real, y pueden llegar a alucinar datos solo por querer darnos una respuesta.

Es así que la mejor forma de introducir un LLM en un sistema de este estilo, es mediante un RAG. Un RAG permite dotar al LLM de información específica de nuestro contexto, y se puede generalizar su uso con cualquier tipo de documentos. Con esto, se soluciona el primer problema, pero aún quedaría el segundo. Para permitir que el sistema pueda tener datos actualizados, hay que darle otra herramienta: el acceso a internet, que será la segunda fuente de información fiable que tendrá el sistema. Con esto, estará provisto de todo lo necesario para responder a nuestras preguntas.

Ahora bien, que pueda responder a las preguntas, no significa que las pueda hacer llegar al usuario. En este punto falta el último paso, que el asistente de verdad se comuniquen de vuelta. De nuevo, la forma de hacerlo es mediante voz, por lo que el sistema deberá implementar un sistema de voz o Text to Speech.

Con esto, el asistente estará dotado de la capacidad de responder a todo en tiempo real, con información fiable y actualizada, y todo ello mediante voz.

CAPÍTULO 3

Desarrollo

En esta sección se redacta como se han implementado todas las partes del sistema planteado en el apartado anterior. Se comentan las tecnologías utilizadas y las dificultades observadas.

El sistema se divide en cuatro bloques principales: Speech to Text, LLM (RAG) con acceso a datos locale y a búsqueda en la web y Text to Speech. A continuación, se explicarán cada uno de ellos.

El apartado de 'búsquedas web' no se comparará ya que es el mismo para ambas modalidades (Tavily) y no tiene ninguna diferencia a nivel de implementación, por lo que se 'evaluará su calidad'.

En los esquemas más abajo no se incluye la interfaz gráfica, ya que esta solo usa el sistema de forma abstracta. Se encarga de gestionar la interacción con el usuario, grabar el audio y reproducir la respuesta final.

Por último, mencionar que la elección de modelos se ha hecho lo más parecida posible entre ambos entornos, para evitar sesgos. Se han exprimido al máximo las capacidades locales, y se han igualado los modelos en la nube a estos para que la comparación sea lo más justa posible. Por supuesto, esto bajo la premisa de la que parte el trabajo: viabilidad y facilidad de implementación además de sin coste económico (directo).

3.1 Implementación Local

Se comentan las tecnologías utilizadas para cada bloque del sistema, y se explica su funcionamiento e integración en un entorno puramente local.

3.1.1. Speech to Text (Whisper)

Para la interpretación de los audios y solucionar la parte del sistema encargada del ASR o Speech to Text, se ha elegido el modelo Whisper [1].

Whisper es un sistema de reconocimiento automático del habla (ASR) desarrollado por OpenAI. Es un modelo muy potente ya que es capaz no solo de transcribir, sino también de realizar traducción e identificar la lengua de forma nativa. Su arquitectura se basa en **Transformers** de tipo **encoder-decoder**, con un entrenamiento exhaustivo sobre miles de horas de audio multilingüe.

El modelo ha demostrado un funcionamiento excelente, resultando muy rápido y fácil de implementar. Se ha observado que, a niveles prácticos, la precisión es prácticamente idéntica en tareas generales, ofreciendo una experiencia sumamente robusta frente a diferentes acentos y ruidos de fondo.

Los únicos fallos detectados han sido cuando se mencionaban palabras muy específicas, como siglas o nombres propios, que el modelo no reconocía y transcribía de forma errónea. Por ejemplo,

al pronunciar "MIARFID", el modelo lo transcribía como "MirFit". O al decir "PyCatan", lo transcribía como "PaiKatan". Estos errores repercuten en cadena, ya que si el asistente recibe la palabra equivocada, la búsqueda RAG rara vez localizará el documento correcto asociado.

Aún así, el rendimiento general del modelo ha sido excelente, con una latencia mínima y una precisión más que suficiente para el propósito del asistente.

3.1.2. RAG (Ollama)

El sistema implementa una arquitectura RAG con recuperación híbrida, es decir, el sistema trata de recuperar de su información local, y en caso de no encontrar la información busca en la web. El proceso se divide en tres fases principales: indexación, evaluación de relevancia y generación. Toda este componente ha sido implementada usando **LangChain** [2].

Indexación y persistencia de datos

Para la gestión del conocimiento local, el sistema utiliza un flujo de procesamiento de documentos PDF basado en **LangChain** [2]. Los documentos son cargados mediante **PyPDFLoader** y fragmentados utilizando **RecursiveCharacterTextSplitter** con un tamaño de chunk de 1000 caracteres y un overlap de 200 caracteres para preservar el contexto semántico entre fragmentos.

Estos fragmentos se transforman en vectores numéricos utilizando modelos de **embeddings** de **Ollama** [3], en concreto se ha utilizado **mistral-nemo** [empty citation], y se almacenan en una base de datos vectorial basada en **ChromaDB**. El sistema incluye una lógica de persistencia que verifica la existencia previa de la base de datos para evitar re-indexaciones innecesarias, es decir, se comprueba si anteriormente se ha generado esta base de datos, y de esta forma se optimiza el tiempo de ejecución.

Recuperación y evaluación de relevancia

Una vez recibida la transcripción del audio, el sistema realiza una búsqueda de similitud en el *vector store* recuperando los $k = 10$ fragmentos más cercanos. Antes de proceder a la generación, se introduce un paso para evaluar la relevancia y comprobar mediante un LLM (**llama3.1**) [4] si el contexto recuperado localmente es suficiente o no para generar la respuesta. Los prompts serían:

```
prompt_evaluador = ChatPromptTemplate.from_messages([
    ("system", "Eres un evaluador de relevancia. Responde estrictamente con una palabra: 'SI' o 'NO'."),
    ("human", f"Pregunta: {pregunta} \n\n Contexto recuperado: {documentos} \n\n ¿Contiene este contexto la informacion necesaria para responder?")
])
```

A partir de la respuesta obtenida tendríamos que:

- Si el contexto recuperado es suficiente para responder a la consulta, se procede con la información local.
- En caso de que el contexto local sea insuficiente o irrelevante, el sistema activa un mecanismo de *fallback* en que utiliza la herramienta **TavilySearchResults** [5] para obtener la información de la web. Para este caso, se ha establecido a 5 el número máximo de resultados que se devolverán por búsqueda y se ha indicado que se realice una búsqueda avanzada.

Generación de respuesta final

La fase final aprovecha el contexto obtenido y lo envía a un modelo para generar la salida (el mismo `llama3.1` en local).

Para distinguir entre la información obtenida de la web y la extraída de los documentos, se dispone de la siguiente lógica de *fallback*:

```
if es_relevante:
    print(" - [OK] Información encontrada en documentos locales.")
    contexto_final = contexto_local
    fuente_info = "tus documentos internos"
else:
    print(" - [!] Información local insuficiente o irrelevante. Consultando internet...")
    web_results = search_tool.invoke({"query": transcription})
    contexto_final = "\n".join([res["content"] for res in web_results])
    fuente_info = "fuentes externas en internet"
```

El *prompt* del sistema está configurado para actuar como un asistente de investigación, obligando al modelo a entregar una respuesta en texto plano, sin formatos Markdown, asegurando una salida natural, fluida y concisa que se adapta a seguir un formato de mensaje de voz. El *prompt* utilizado es:

```
system_prompt = (
    "Eres un asistente de investigacion. Responde de forma natural, fluida y concisa, no des  

    ↪ detalles innecesarios y ves directamente al grano. "  

    f"La informacion proviene de {fuente_info}. "  

    "IMPORTANTE: No uses negritas, ni asteriscos, ni listas, ni ningun formato markdown. "  

    "Escribe todo en un parrafo o parrafos de texto plano, como si fuera una carta o un  

    ↪ mensaje de voz."  

    "\n\nContexto: {context}"
)

prompt_final = ChatPromptTemplate.from_messages([
    ("system", system_prompt),
    ("human", "{input}"),
])
```

3.1.3. Text to Speech (Kokoro)

Una vez que el sistema RAG ha generado la respuesta de texto, lo único que queda es comunicar el feedback al usuario mediante voz en local.

Para ello, se ha integrado Kokoro [6], un modelo de síntesis de voz (TTS) de código abierto que destaca por su gran eficiencia. Está basado en una arquitectura híbrida que usa conceptos de *StyleTTS2* [7] e *ISTFTNet* [8]. Al tener únicamente 82 millones de parámetros, usa una red neuronal muy ligera para predecir el estilo y la duración de los fonemas.

Gracias a este diseño compacto, consigue generar audio en menos del tiempo que cuesta reproducirlo, con una latencia mínima. A nivel de recursos, apenas ocupa memoria en la GPU, dejando espacio libre para el resto del sistema (como el LLM o el reconocimiento de voz).

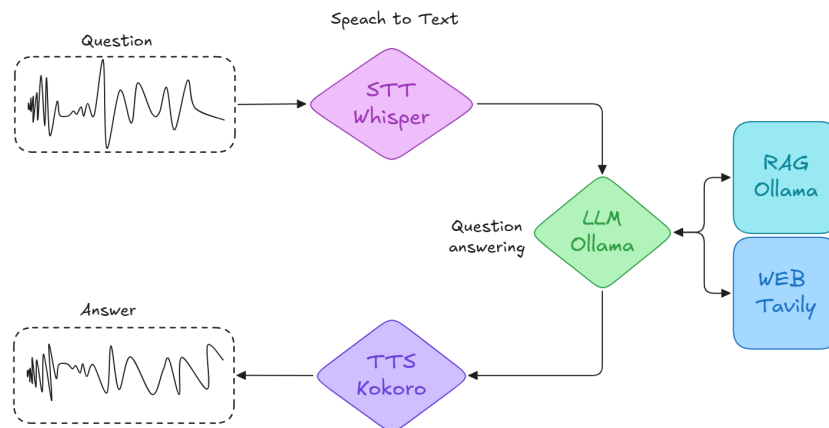


Figura 3.1: Estructura del sistema del asistente de voz usando modelos *locales* para cada fase.

3.2 Implementación Cloud

Dado que la integración paso a paso de los componentes ha sido muy similar a la abordada en la sección local, no se volverá a explicar su funcionamiento a profundidad, se ha sustituido el grueso de las librerías locales por llamadas y peticiones API a los proveedores, reutilizando la misma lógica de orquestación y *fallback* entre fuentes de información. Para el RAG los mismos prompts etc.

Inicialmente, el planteamiento ideal era concentrar y delegar todas las tareas (STT, LLMs, TTS) a un único servicio, en este caso AWS. Unificar pretendía simplificar la gestión e integración, pero rápidamente emergieron problemas. Se intentaron usar los servicios completos de AWS, pero se encontraron trabas limitantes a nivel de validación de cuentas, suscripciones bloqueadas y fallos donde el servicio simplemente no funcionaba tras el alta de la tarjeta. Aunque el pago y validación deberían ser un trámite rápido, las demoras burocráticas del proveedor forzaron la paralización de la solución centralizada.

A consecuencia de estas restricciones e impedimentos, se hizo imperativo diversificar y orquestar el sistema como un collage de distintos proveedores externos para cada fase:

- **GROQ:** Se empleó para el uso del LLM que gestiona y genera la lógica conversacional.
- **Hugging Face:** Para la obtención de los modelos de *embeddings* para el RAG.
- **Tavily:** Se recurrió a esta API para las búsquedas web.
- **AWS (limitado):** Tras sortear algunas barreras iniciales, logramos configurarlo al menos como el proveedor final de *Text to Speech* (TTS).

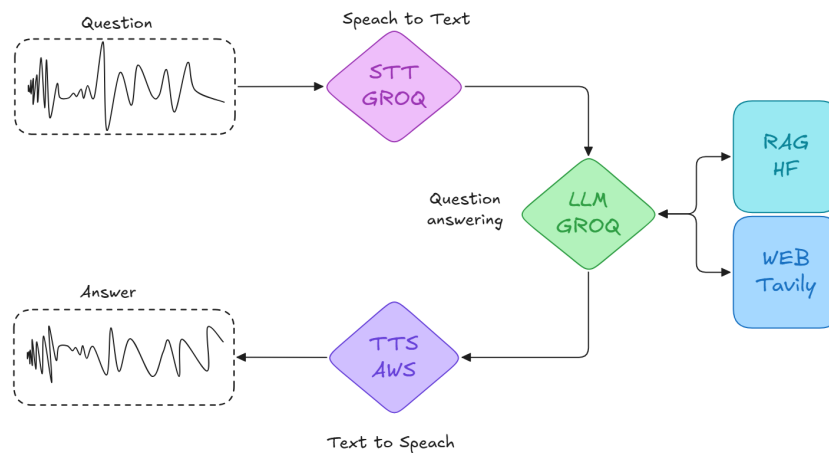


Figura 3.2: Estructura del sistema del asistente de voz usando proveedores *cloud* para cada fase.

3.3 Interfaz gráfica

Para el desarrollo de la interfaz gráfica, se ha hecho uso de la librería `Streamlit` [9].

A partir de la documentación de esa y con ayuda de LLMs, se generó una plantilla base por la que empezar y poder integrar el sistema a nuestra pipeline. Se han ajustado las rutas y se han añadido opciones para elegir qué configuración (`Local` o `cloud`) utilizar.

Con todo esto, se puede lanzar una interfaz gráfica muy sencilla que permite grabar o subir preguntas en tiempo real. A los pocos segundos, se obtiene una respuesta por voz.

CAPÍTULO 4

Resultados y Análisis

Para poder evaluar y cuantificar la competitividad del entorno local frente a las soluciones en la nube, se llevaron a cabo una serie de pruebas sistemáticas. Se diseñó un experimento que parte de la grabación previa de 30 audios con preguntas y peticiones variadas, obteniendo un total de 60 ejecuciones evaluadas: 30 de cada modalidad (local y cloud).

4.1 Metodología de Evaluación

La recogida de métricas se segmentó en distintas áreas para analizar de manera global el desempeño del sistema. Se recopilaron los siguientes parámetros:

- **Tiempos de ejecución:** Se midieron de forma automatizada insertando marcas en el código. El tiempo total por petición se describe matemáticamente como la suma de sus fases: $T_{total} = T_{STT} + T_{RAG} + T_{TTS}$
Ya que la búsqueda web es la misma en ambas modalidades, no se incluye en esta fórmula para evitar sesgos, aunque se tiene en cuenta su impacto en el tiempo de respuesta final.
- **Precisión del STT (WER):** Para verificar qué tan correcta era la transcripción del audio comparado con el texto real, se empleó la métrica *Word Error Rate*. Se estandarizaron internamente ambas cadenas usando el módulo `werpy.normalize` [**<empty citation>**] que, en esencia, consiste en secuenciar las frases pasando todo a minúsculas, eliminando los signos de puntuación, borrando los espacios sobrantes y homogeneizando los caracteres especiales.
- **RAG Matching Source:** Esta métrica dictamina, de forma binaria (0 ó 1), si el sistema recurrió correctamente a la fuente de información adecuada. Si el dato necesario estaba disponible localmente y el sistema usaba esa base, sumaba 1. Si consultaba internet ignorándolo, puntuaba 0 y viceversa con la información Web.
- **RAG Matching Response:** Valora si el sistema devolvió o no la contesta correcta final independientemente de la fuente de donde provino (o si lo dedujo de su conocimiento interno). Es aquí donde se detectó el impacto de la fuente comentado previamente: si el sistema infiere que *no* dispone del contexto y sale a la web, aunque no sea lo ideal, es muy posible que de la respuesta correcta. Por desgracia en la contraparte, cuando el sistema evalúa erróneamente que *sí* tiene la respuesta en el documento y se aísla, el LLM padece de la escasez del contexto y termina forzando una alucinación (es decir, proporcionando un dato incorrecto o inventado).
- **TTS Score:** Debido a la subjetividad del paradigma conversacional, se empleó una escala manual del 0 al 5 valorando parámetros humanos tales como la fluidez, pausas lógicas, gusto general y naturalidad a la hora de escuchar la respuesta devuelta por los altavoces. En esencia, se evaluó según el ‘como les pareció a los evaluadores’.

4.2 Extracción de Resultados

La tabla 4.1 y el gráfico ilustrativo en la figura 4.1 encapsulan el resumen de datos cosechados a través de las 60 ejecuciones analizadas en ambas modalidades de inferencia.

Tabla 4.1: Métricas obtenidas en la evaluación de los 30 audios.

Métrica	Local	Cloud
Transcribe Time (s)	3.95	0.41
RAG Time (s)	6.04	31.98
TTS Time (s)	1.03	0.86
Total Time (s)	11.02	33.25
Transcription WER (%)	5.22 %	6.32 %
RAG Matching Source (%)	80.00 %	90.00 %
RAG Matching Response (%)	63.33 %	83.33 %
RAG Matching Source (Raw)	24 / 30	27 / 30
RAG Matching Response (Raw)	19 / 30	25 / 30
TTS Score (0-100)	50	90

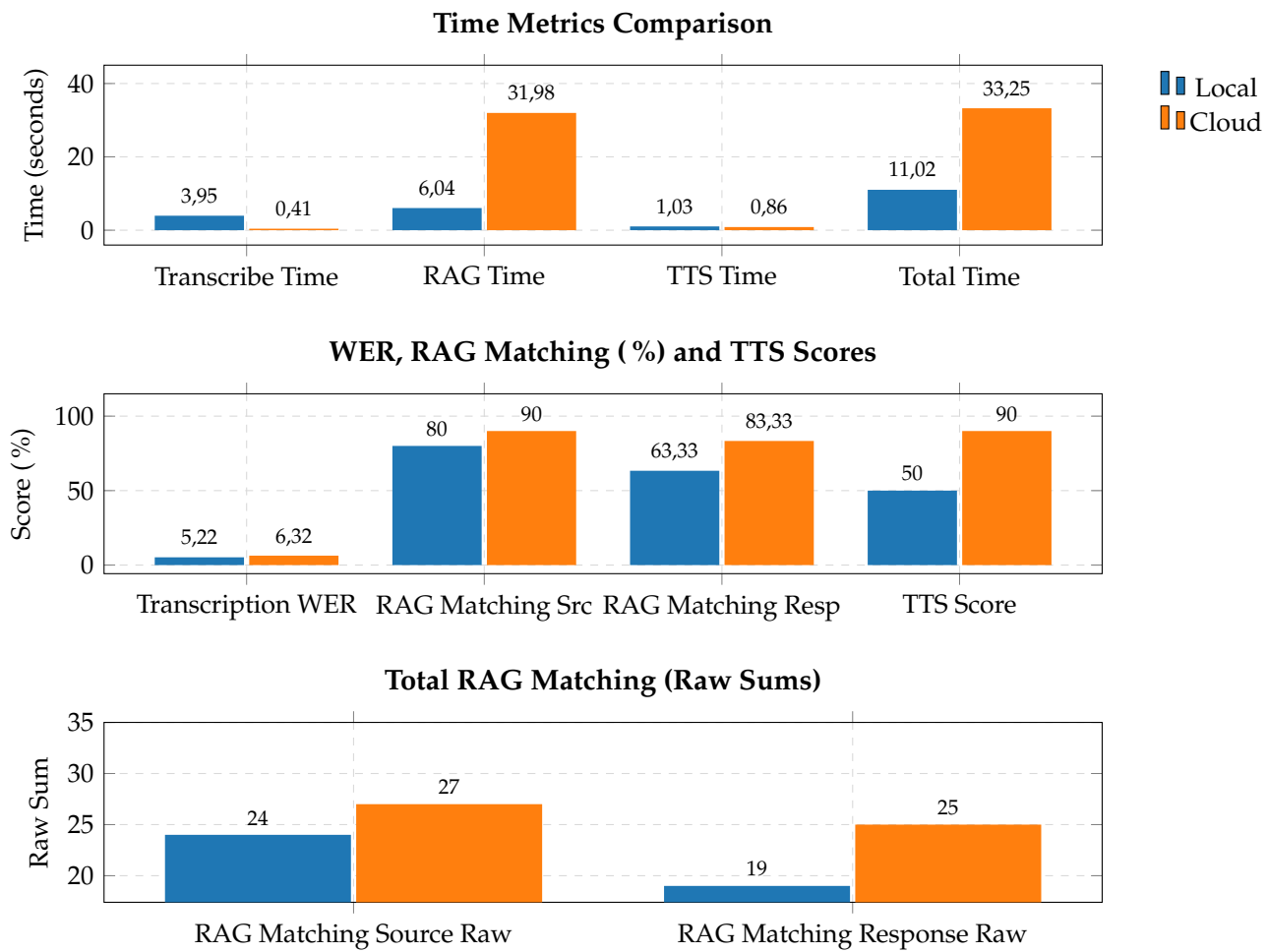


Figura 4.1: Visualización gráfica del contraste de resultados (Tiempos, % de acierto y Conteo Crudo).

Como se aprecia fuertemente en las franjas de tiempos, el entorno en la nube resulta estar enormemente penalizado en cuanto a latencia de respuesta global (principalmente por el excesivo tiempo consumido en la orquestación RAG). Pese a que en tareas aisladas como la transcripción de audio (STT) la nube demuestra ser notablemente más rápida frente a la ejecución local, el tiempo final se resiente.

Sin embargo, la fluidez del habla y en el porcentaje de entendimiento de búsqueda (Match Response). Por otro lado, la diferencia en la correcta ingesta de las palabras (STT WER) fluctúa por la mínima con un excelente resultado en ambos entornos, resultando una discrepancia poco representativa para la experiencia de un usuario final.

CAPÍTULO 5

Comparativa implementación

A lo largo de este y el siguiente capítulo, se plantea una comparativa entre las dos modalidades de implementación del asistente. Se analizarán las diferencias en cuanto a rendimiento, calidad de respuesta, facilidad de desarrollo y complejidad de mantenimiento.

5.1 Modelos STT

Tanto en la versión local como en la nube, la integración del sistema de reconocimiento de voz (STT) ha resultado en un proceso casi *plug and play* en ambas opciones. Sin embargo, a nivel de rendimiento sí se observa una diferencia importante: el procesamiento externalizado a la nube es considerablemente más rápido. A pesar de esto, ambos sistemas transcriben a una velocidad sumamente alta que no supone un factor de cuello de botella crítico para la viabilidad del flujo en general.

5.2 RAG y Flexibilidad del Ecosistema

En cuanto al componente RAG, es importante puntualizar el grado de facilidad con el que se adaptó el sistema de una modalidad a otra. Con la misma implementación de código para una opción, se pudo introducir la otra directamente.

LangChain permite utilizar tanto los modelos locales (vía Ollama) como los LLMs y embeddings de proveedores externos, abstrayendo estas diferencias bajo la misma interfaz. De este modo, en el desarrollo final, con instanciar el modelo apuntando al servicio correspondiente, el RAG completo ya estaba funcionando.

5.3 Modelos TTS

En la fase de síntesis de voz, se puede observar una gran diferencia entre Kokoro (modelo local) y el uso de AWS como modelo *cloud*.

La principal desventaja del modelo local es el resultado final: aunque la voz es clara y se entiende perfectamente, carece un poco de 'personalidad' y puede sonar ligeramente plana en comparación con las voces de última generación.

A diferencia de lo que se podría intuir en un principio, el uso de la alternativa en la nube aporta resultados que superan con fluidez al modelo local. La voz generada por los servicios web es mucho más natural, incluyendo pausas lógicas e inflexiones. Lo más llamativo de esta comparativa es que esta inmensa mejora en la naturalidad se consigue a cambio de una latencia apenas superior a la de Kokoro.

Tras analizar ambas opciones, el compromiso entre calidad y latencia se hace evidente pero menos marcado de lo esperado. Mientras que Kokoro (local) proporciona inmediatez, el modelo en la nube ofrece un audio sobresaliente que compensa la poca espera. La elección final depende de que tan vital sea la comodidad de una voz casi totalmente humana y una latencia algo mayor.

5.4 Complejidad de Integración Múltiple

Si se delega 'la computación' en 'otros', la implementación teóricamente debería ser más sencilla: sin gestión de dependencias y sin preocupaciones por el mantenimiento del hardware. Sin embargo, recurrir a proveedores *cloud* añade un grado de complejidad técnica y burocrática notable al proceso de desarrollo una vez que la cuenta única falló.

Desde una perspectiva crítica, ensamblar estas piezas dispares es bastante engorroso frente al modelo monolítico local. Cada proveedor *cloud* dicta sus propios mecanismos de autenticación (API Keys), estructuras de conexión, cuotas de uso y esquemas de precios. Toda esta fragmentación eleva la complejidad de despliegue y la del desarrollo del código, a pesar de que su latencia y respuesta sea altamente competente.

5.5 Curiosidades y Fallos Observados

Durante las fases de pruebas del sistema completo, salieron comportamientos inusuales y limitaciones prácticas que conviene destacar, ya que muestra los sesgos y el margen de error presente tanto en los sistemas generativos como en los modelos de transcripción.

Por un lado, respecto al reconocimiento de voz (STT), se detectaron limitaciones fonéticas al intentar transcribir siglas o nombres propios muy específicos del dominio. Por ejemplo, al pronunciar de forma natural las siglas del máster en concreto ("MIARFID"), el modelo fallaba repetidamente forzando la transcripción a la palabra inventada "MirFit." similar. Ocurría un fenómeno idéntico al nombrar el repositorio del "PyCatan", el cual terminaba siendo transcrito como "PaiKatan". Estos fallos repercuten en cadena; si el asistente recibe la palabra equivocada, la búsqueda RAG rara vez localizará el documento correcto asociado, y si el LLM no es capaz de inferir el término correcto, la respuesta final se ve comprometida.

Por otro lado, se observó una inconsistencia notable en la arquitectura RAG a la hora de determinar correctamente la fuente a consultar. En ocasiones, la respuesta a una pregunta se encontraba perfectamente indexada en los ficheros locales. Sin embargo, el LLM evaluador (llama3.1) decidía que el contexto local era insuficiente o irrelevante, forzando un *fallback* hacia la búsqueda web en internet: Sobre todo en preguntas sobre Messi.

Luego por ejemplo, si las 'fuentes de la verdad' no son fiables o varían, como algunas veces pasa en internet, no podemos asegurar respuestas consistentes o correctas. Si el sistema hace uso de la búsqueda web, y esta devuelve información errónea, el modelo generará una respuesta incorrecta, aunque el proceso de recuperación haya funcionado correctamente. Un ejemplo ha sido la pregunta '¿Cuál es la raza de perro más peluda según los expertos?' a lo que cada ejecución (cloud o local), respondía aleatoriamente con 'Chow Chow', 'Akita' o 'Pulli'... (véase la Figura 5.1).

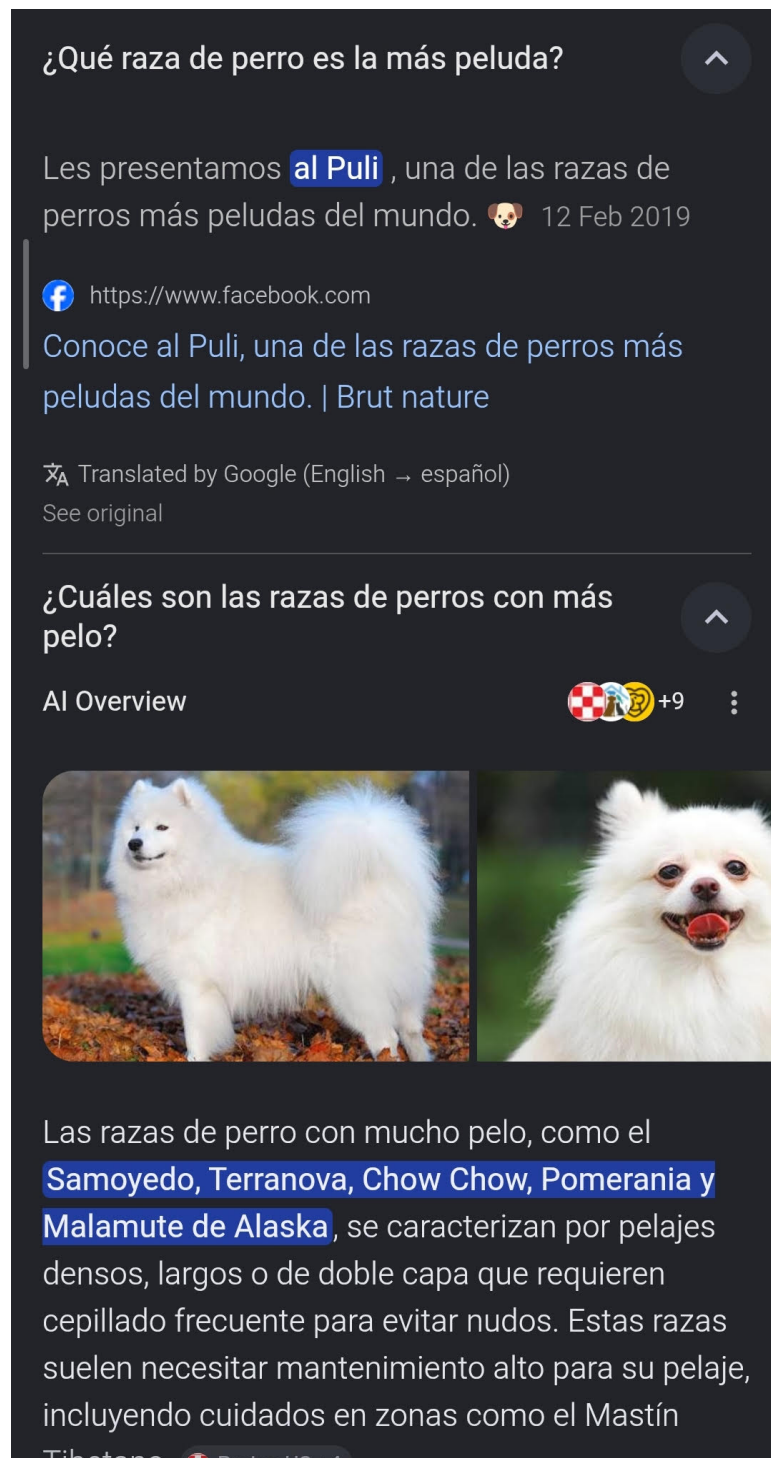


Figura 5.1: Ejemplo de inconsistencia en el sistema RAG, donde el evaluador prefiere buscar en la web ignorando los datos locales explícitamente disponibles.

5.6 Requisitos Computacionales, privacidad y elección de modelos

Donde realmente recae el punto clave a la hora de decidir entre abordar una implementación puramente local o externalizar su computación a la nube, es en los requisitos físicos de la máquina y la privacidad.

Alojar LLM o transformers de gran tamaño como algunas versiones de Whisper, implica requerimientos considerables de VRAM para la GPU y memoria RAM. Además de estar de depender

totalmente de agentes externos, pasando por el riesgo de externalizar tus datos a terceros, lo que puede ser un gran inconveniente para muchos usuarios.

Por ello, si se dispone del hardware necesario para sostener las ejecuciones de manera fluida y concurrente, la versión local es una muy buena opción. De lo contrario, si las capacidades de la máquina están limitadas, se puede suplir este déficit usando las APIs de los proveedores externos *cloud*, garantizando una ejecución sin cuellos de botella y con un catálogo mucho mayor de modelos y servicios disponibles.

Se recuerda que la comparativa se ha hecho usando los modelos más parecidos posibles, expresando al máximo la capacidad local e igualando los modelos en la nube a estos para evitar sesgos. Sin embargo, en un entorno real, la elección de modelos sería mucho más flexible y se podrían usar versiones más potentes en la nube que no podrían ser ejecutadas localmente, lo que aumentaría aún más la brecha de calidad entre ambas opciones.

Además de esto, se han realizado todas las ejecuciones cloud con suscripciones gratuitas de los proveedores, SIN haber llegado al tope de uso. Esto implica dos cosas:

1. Sin hacer inversión en hardware, conseguimos un sistema más que competente al local.
2. Si el usuario quisiera, podría abrir la puerta de las suscripciones de pago, lo que le permitiría usar modelos aún más potentes y con mejores resultados, aumentando la brecha de calidad frente a la versión local. Sobre todo con respecto a los LLMs.

De nuevo, a cambio de una dependencia total de terceros y la externalización de datos,

CAPÍTULO 6

Conclusiones y trabajos futuros

Tras finalizar el desarrollo e implementación del asistente, se pueden extraer varias conclusiones importantes sobre el estado actual de estas tecnologías y su integración.

En primer lugar, ha quedado demostrado que hoy en día es totalmente viable crear un flujo completo de interacción por voz en una máquina personal. Hace poco tiempo, combinar transcripción, búsqueda de información, razonamiento y síntesis de voz en segundos era algo reservado a grandes empresas. Ahora, con el hardware adecuado, se puede procesar una pregunta y obtener una respuesta hablada en un tiempo muy razonable.

Respecto a los componentes del sistema, se han observado diferencias notables en su complejidad:

- **El reconocimiento de voz (STT)** es, a día de hoy, un problema prácticamente resuelto para tareas generales. Herramientas como Whisper funcionan casi como un 'plug and play': se conectan y funcionan. Para idiomas comunes como el español o el inglés, no hace falta reentrenar ni realizar fine-tuning; el modelo entiende lo que se dice con una precisión excelente desde el primer momento.
- **La fiabilidad de la información** sigue siendo el punto más delicado. Se sabe que un LLM por sí solo no es suficiente, ya que tiende a 'alucinar' o inventar datos. Por tanto, es crucial que el sistema tenga acceso a fuentes externas (mediante RAG para documentos locales o búsqueda web) para poder confiar en sus respuestas. Tal y como revelan las métricas de evaluación insertadas, aunque el modelo *cloud* obtiene un ratio de resolución superior (83 % de aciertos limpios), confiar ciegamente en fuentes locales mal evaluadas degenera drásticamente en alucinaciones (estancando el *benchmark* local en un 63 %). Sin esta conexión dinámica entre recursos asimilados, el asistente puede ser elocuente pero peligroso.
- **Compromiso entre Velocidad y Naturalidad en Arquitecturas:** Enlazando con el análisis métrico del capítulo anterior, se constata que delegar las operaciones vitales a un monolito local permite un altísimo desempeño en latencia pura (11,02s), logrando un entorno casi fluido. Sin embargo, trasladarlo al orquestado de microproveedores *cloud* multiplica la latencia notablemente (33,25s). Aun con esta penalización temporal general, es este último enfoque en la nube el que ostenta la victoria imbatible en cuanto a naturalidad final de la síntesis de voz, promediando un 4,5/5 frente al modesto 2,5 del modelo local. La síntesis de voz será vital y su elección dependerá subjetivamente del nivel de humanidad buscado por el consumidor final.

Hay que mencionar que en aplicaciones reales, estos modelos deberían ser usados de otra forma. Por ejemplo, en lugar de esperar a hacer toda la generación de texto para luego hacer la voz, se podrían usar técnicas de *streaming* para empezar a reproducir el audio mientras se sigue generando la respuesta del LLM. Esto reduciría la latencia percibida, difuminando de alguna forma los picos temporales identificados (especialmente en la variante *cloud*) y mejoraría exponencialmente la experiencia de usuario.

Independientemente de si se utiliza la versión cloud o la local, tras analizar las 60 respuestas generadas por los usuarios, no se considera que ninguna de las dos opciones ofrezca una respuesta de RAG claramente mejor a nivel cualitativo. Ambas opciones proporcionan resultados coherentes. Sin embargo, observando las métricas obtenidas con detalle, se aprecian ciertas diferencias de rendimiento y precisión que podrían inclinar la balanza dependiendo de la disponibilidad de hardware.

6.1 Trabajos futuros

Aunque el sistema es funcional, quedan muchas opciones para mejorarlo y convertirlo en un producto más acabado:

- **Sistema 'siempre escuchando' (Wake Word):** Actualmente, la interacción requiere una acción manual (grabar/enviar). El siguiente paso es implementar un sistema de detección de palabras clave (como 'Oye Charo') que mantenga el micrófono abierto en bajo consumo y solo active el proceso pesado cuando se le llame.
- **Implementación en sistemas empujados:** Mover el sistema de un PC de escritorio a un hardware dedicado y pequeño, como una Raspberry Pi o una NVIDIA Jetson. Esto permitiría tener el asistente en cualquier lugar de la casa sin depender de un ordenador encendido, acercándose más al concepto de 'altavoz inteligente'. Dependiendo del hardware, habría que delegar servicios a la nube.
- **Ampliación del agente:** Mejorar la capacidad del RAG para usar volúmenes de datos mucho mayores o añadirle más herramientas MCP, aumentaría las capacidades del sistema, y no lo limitaría a preguntas y respuestas. Ahora, esto podría requerir de modelos de lenguaje más grandes.
- **Streaming de respuesta:** Como se ha comentado, implementar un sistema de streaming tanto para la generación de texto como para la síntesis de voz mejoraría mucho la experiencia de usuario, reduciendo la latencia percibida.

Bibliografía

- [1] Alec Radford et al. *Robust Speech Recognition via Large-Scale Weak Supervision (Whisper)*. 2022. URL: <https://github.com/openai/whisper>.
- [2] LangChain, Inc. *LangChain Documentation*. Accessed: 2025-01-25. 2025. URL: <https://docs.langchain.com/>.
- [3] Jeffrey Morgan y Ollama Team. *Ollama: Get Up and Running with Large Language Models locally*. Accessed: 2026-01-25. 2023. URL: <https://ollama.com/>.
- [4] Meta AI. *Llama 3.1 Model Card*. Accessed: 2026-01-25. Jul. de 2024. URL: <https://huggingface.co/meta-llama/Llama-3.1-8B>.
- [5] Tavily AI. *Tavily: The Search Engine for AI Agents*. Accessed: 2026-01-25. 2024. URL: <https://tavily.com/>.
- [6] hexgrad. *Kokoro-82M: A Low-Parameter TTS Model*. Accessed: 2025-01-25. 2025. URL: <https://huggingface.co/hexgrad/Kokoro-82M/blob/main/VOICES.md#spanish>.
- [7] Yinghao Aaron Li et al. «StyleTTS 2: Towards Human-Level Text-to-Speech through Style Diffusion and Adversarial Training with Large Speech Language Models». En: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 36. NeurIPS 2023. 2023. URL: <https://arxiv.org/abs/2306.07691>.
- [8] Takuhiro Kaneko et al. «iSTFTNet: Fast and Lightweight Mel-Spectrogram Vocoder Incorporating Inverse Short-Time Fourier Transform». En: *ICASSP 2022 - 2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 2022, págs. 6207-6211. DOI: [10.1109/ICASSP43922.2022.9746529](https://doi.org/10.1109/ICASSP43922.2022.9746529). URL: <https://arxiv.org/abs/2203.02395>.
- [9] Snowflake Inc. *Streamlit: A Faster Way to Build and Share Data Apps*. Accessed: 2025-01-25. 2025. URL: <https://streamlit.io/>.